AN INTELLIGENT SELF-HEALING OPERATING SYSTEM DIGITAL TWIN WITH PREDICTIVE PERFORMANCE ANALYTICS

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0402-Operating Systems

to the award of the degree of

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING

Submitted by

J. Renuka Devi (192573007)

Under the Supervision of

Dr. Priskilla Angel Rani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “AN INTELLIGENT SELF-HEALING OPERATING SYSTEM DIGITAL TWIN WITH PREDICTIVE PERFORMANCE ANALYTICS” has been carried out by J. Renuka Devi (192573007) , Bhavana(192524178), Prakeerthi(192524279), Ramya(192572163) under the supervision of Dr. Priskilla Angel Rani and is submitted in partial fulfilment of the requirements for the current semester of the BE program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.V.R. Vimal

Professor

Department of Computer Science and Engineering

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.Priskilla Angel Rani

Professor

Department of Computer Science and Engineering

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, J. RenukaDevi(192573007) , Bhavana(192524178), Prakeerthi(192524279), Ramya(192572163) of the CSE SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "AN INTELLIGENT SELF-HEALING OPERATING SYSTEM DIGITAL TWIN WITH PREDICTIVE PERFORMANCE ANALYTICS" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: SIMATS Chennai

Date:

Signature of the Students with Names

J. Renuka Devi (192573007)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Student 1	Self-healing, fault recovery and system-level integration	Developed fault detection, recovery decision logic and self-healing workflow	Tested fault scenarios, recovery responses and module integration	Ch. 3, Ch. 4 & Ch. 5	**25%**
Student 2	System Monitoring & Digital Twin	Developed CPU, memory, thread monitoring and system-state representation	Tested metric collection and system health monitoring	Ch. 1 & Ch. 3	**25%**
Student 3	Predictive Analytics & Anomaly Detection	Developed performance analysis, trend analysis and anomaly detection	Tested analytical results and performance trends	Ch. 2, Ch. 3 & Ch. 4	**25%**
Student 4	Database, Dashboard & Security	Developed MySQL/JPA persistence, REST integration, dashboard and authentication	Tested database operations, APIs and dashboard functionality	Ch. 4 & Ch. 5	**25%**
Total					**100%**

ABSTRACT

Modern computing systems continuously experience variations in CPU utilisation, memory consumption, thread activity and application workload. Conventional monitoring approaches generally identify abnormal conditions but depend on manual intervention or predefined administrative actions to restore normal operation. Such approaches can increase response time and reduce system availability, particularly when multiple performance indicators interact.

This project presents an Intelligent Self-Healing Operating System Digital Twin with Predictive Performance Analytics, a software-based prototype designed to continuously represent the operational state of a host computing system and support automated fault identification and recovery. The proposed architecture combines system monitoring, digital-twin state representation, predictive performance analytics, fault detection, self-healing recovery, persistent logging and an interactive dashboard.

The monitoring layer collects operating-system-related performance indicators such as CPU utilisation, memory usage, active threads and processor information using Java management interfaces. The collected measurements are analysed to determine system health and identify abnormal operating conditions. Predictive analytics and trend-based analysis are incorporated to identify performance deterioration before it becomes a critical condition. When a fault is detected, the self-healing module classifies the fault and selects a predefined, controlled recovery strategy. Recovery outcomes are recorded in the database and made available through REST APIs and the dashboard.

The system is implemented using Java, Spring Boot, Spring Data JPA, MySQL, HTML, CSS and JavaScript, with a modular backend architecture. The prototype is evaluated through functional, integration and fault-simulation testing. The resulting system demonstrates an integrated workflow from system-state acquisition through analysis, fault identification, recovery and historical monitoring.

Keywords: Digital Twin, Self-Healing Systems, Operating System Monitoring, Predictive Analytics, Fault Detection, System Resilience

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Abbreviation	Description
OS	Operating System
DT	Digital Twin
CPU	Central Processing Unit
RAM	Random Access Memory
API	Application Programming Interface
REST	Representational State Transfer
JPA	Java Persistence API
JVM	Java Virtual Machine
ML	Machine Learning
AIOps	Artificial Intelligence for IT Operations
DB	Database
UI	User Interface
SQL	Structured Query Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern computer systems operate under continuously changing workloads. CPU utilisation, memory consumption, thread activity and application demand can vary significantly during normal operation. A sudden increase in resource utilisation may result in degraded performance, delayed application responses and reduced system availability.

Traditional system monitoring tools are primarily designed to observe and report system conditions. Although monitoring provides valuable information, detection alone does not automatically provide an appropriate response to an emerging fault. A human administrator or external management process may still be required to interpret the condition and initiate corrective action.

The concept of a Digital Twin (DT) provides a mechanism for maintaining a software representation of a physical or computational system. In the proposed project, the digital twin represents the current operational state of the host computing environment using continuously collected performance measurements. Digital-twin research has increasingly combined real-time monitoring, modelling, analytics and prognostics to improve system reliability and maintenance decisions.

The project extends this concept by integrating the digital representation with predictive performance analysis and self-healing mechanisms. Instead of treating monitoring, prediction and recovery as isolated activities, the proposed system connects them into a unified workflow:

System Monitoring → Digital Twin State → Predictive Analytics → Fault Detection → Self-Healing → Logging → Dashboard

The resulting architecture is intended to demonstrate how intelligent software systems can identify abnormal operating conditions, determine an appropriate response and maintain an auditable record of system behaviour.

1.2 Need for the Project

The project is motivated by four major engineering requirements.

1.2.1 Increasing system complexity

Modern software systems contain multiple interacting processes, threads and resource-intensive services. A fault in one component can influence overall system performance.

1.2.2 Limitations of monitoring-only systems

A monitoring system can identify high CPU or memory usage, but identification alone does not constitute recovery. An intelligent system should be capable of interpreting the condition and initiating an appropriate controlled response.

1.2.3 Requirement for proactive analysis

Reactive fault handling occurs after a problem becomes visible. Trend analysis and predictive analytics can provide an earlier indication of performance degradation and therefore support proactive decision-making.

1.2.4 Requirement for system resilience

A resilient computing environment should detect abnormal behaviour, respond consistently and maintain information about the event. Self-healing mechanisms address this requirement by connecting fault detection with predefined recovery actions.

The proposed system therefore addresses the engineering need for a unified framework that combines observability, prediction, fault handling and recovery.

1.3 Problem Statement

Existing system monitoring approaches primarily provide visibility into resource utilisation but do not necessarily integrate continuous system-state representation, predictive performance analysis, automated fault classification and controlled recovery within a single software architecture.

The engineering problem addressed by this project is:

To design and implement a software-based operating-system digital twin capable of representing system performance, analysing resource behaviour, identifying abnormal conditions and initiating controlled self-healing responses while maintaining persistent records of system state, faults and recovery actions.

The problem involves multiple interacting factors including CPU utilisation, memory consumption, thread activity, system health, fault severity, recovery decisions and historical performance.

1.4 Project Aim

The aim of the project is:

To develop an intelligent self-healing operating system digital twin that monitors system performance, analyses operational trends, detects faults and performs controlled recovery actions while providing real-time visualisation and persistent system records.

1.5 Project Objectives

- To design a modular architecture for an operating-system digital twin.
- To implement real-time collection of CPU, memory, thread and processor information.
- To develop a digital-twin state representation reflecting the current operational condition of the host system.
- To implement predictive and trend-based performance analytics for identifying abnormal system behaviour.
- To develop a fault detection mechanism for CPU, memory and thread-related performance conditions.
- To design and implement a self-healing mechanism that selects controlled recovery actions based on detected fault conditions.
- To maintain persistent system metrics, fault records and recovery logs using MySQL and JPA.
- To develop REST APIs and an interactive dashboard for real-time monitoring and historical analysis.
- To verify the proposed system using functional, integration and controlled fault-simulation tests.
- To evaluate the effectiveness and limitations of the proposed architecture using measured system behaviour.

1.6 Scope

Included

The project includes:

- System performance monitoring.
- CPU utilisation monitoring.
- Memory utilisation monitoring.
- Active-thread monitoring.
- Processor/core information.
- Digital-twin state representation.
- Threshold-based health classification.
- Trend and predictive performance analysis.
- Fault classification.
- Controlled self-healing/recovery logic.
- Recovery status reporting.
- Persistent database logging.
- REST APIs.
- Interactive dashboard.
- Authentication interface.
- Historical monitoring and recovery records.
- Controlled fault simulation for testing.

Excluded

The prototype does not perform unrestricted operating-system modification or destructive remediation. It does not intentionally terminate arbitrary processes, modify protected operating-system files, restart the host machine or perform kernel-level modifications.

The recovery mechanism is designed as a controlled academic prototype, where recovery actions are predefined and safe to execute.

Assumptions

- The application has permission to access the performance information exposed by the Java runtime and operating system.
- The backend executes on the system whose metrics are being monitored.
- MySQL is available during database-dependent operation.

- The monitored metrics are sufficiently representative for prototype-level system health analysis.

Boundary Conditions

The system monitors the environment accessible to the running backend application. Therefore, the collected metrics represent the host system on which the backend is executing, rather than an arbitrary remote computer.

1.7 Expected Engineering Outcome

The expected outcome is a functional software prototype integrating:

Host System → Monitoring → Digital Twin → Predictive Analytics → Fault Detection → Self-Healing → Database → Dashboard

The final system is expected to provide real-time system-state information, identify abnormal performance conditions, perform predefined recovery actions, record system events and provide visual access to current and historical information.

The project is intended as a prototype demonstrating the engineering feasibility of integrating digital-twin concepts with predictive analytics and self-healing mechanisms.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review focused on recent research related to four technical areas:

- Digital twin technology.
- Predictive analytics and prognostics.
- Fault detection and diagnosis.
- Self-healing and AIOps-based incident management.

Peer-reviewed research published primarily between 2023 and 2024 was examined using academic publication sources. Research papers were selected based on their relevance to digital twins, machine-learning-assisted monitoring, fault diagnosis, self-healing systems and intelligent IT operations.

2.2 Review of Existing Work

Xiao et al. (2024)

Xiao et al. investigated digital-twin-driven prognostics and health management for industrial assets. Their work demonstrates the role of digital twins in representing assets and supporting fault-related analysis, reliability improvement and maintenance decisions. The research establishes a strong relationship between digital-twin technology and predictive health management.

Relevance: The proposed project adopts the same general principle of connecting a digital representation with health/performance analysis, but applies it to an operating-system monitoring context.

Nele et al. (2024)

Nele et al. presented a multi-scale review of machine learning applications in digital-twin technology. The study discusses how machine learning can support modelling, monitoring, optimisation and intelligent decision-making within digital-twin environments.

Relevance: This supports the integration of predictive analytics into the proposed digital twin rather than treating the twin as only a visual representation.

Aldrini, Chihi and Sidhom (2024)

Aldrini et al. reviewed fault diagnosis and self-healing techniques for smart manufacturing. Their study covers fault detection, diagnosis and self-healing/fault-tolerant strategies and demonstrates the importance of combining fault identification with appropriate recovery mechanisms.

Relevance: The proposed recovery architecture follows the broader principle of connecting fault diagnosis to predefined recovery behaviour.

Remil et al. (2024)

Remil et al. reviewed AIOps solutions for incident management and discussed detection, prediction, root-cause analysis and automated healing actions. Their review highlights the potential of intelligent operational systems to move from manual incident management towards automated and data-driven responses.

Relevance: The proposed project applies similar concepts at a prototype operating-system level by integrating monitoring, analytics, fault handling and recovery.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Conventional system monitoring	Simple and effective resource observation	Usually monitoring-focused; limited automated recovery	Provides foundation for system metrics
Digital-twin-based PHM	Combines system representation and predictive health analysis	Often developed for industrial assets	Supports digital-twin + predictive approach
ML-enabled digital twins	Enables intelligent analysis and prediction	ML models can require substantial data and validation	Supports predictive analytics
Fault diagnosis/self-healing systems	Connects fault identification with recovery	Often domain-specific	Supports recovery architecture
AIOps incident management	Supports detection, prediction and automated operational actions	Often designed for large IT infrastructures	Provides conceptual basis for intelligent operations
Proposed system	Integrates monitoring, digital twin, analytics,	Prototype-level predictive and recovery capabilities	Directly addresses the identified integration gap

	fault detection, recovery, database and dashboard		

2.4 Research / Engineering Gap

The reviewed literature demonstrates significant progress in digital twins, predictive analytics, fault diagnosis and self-healing systems. However, these capabilities are frequently studied as separate technical areas or within specialised domains such as industrial manufacturing and large-scale IT operations.

A practical gap therefore exists in developing a compact, modular and demonstrable software architecture that combines system-state representation, performance analytics, fault classification, controlled self-healing, persistent logging and visualisation for an operating-system environment.

The proposed project addresses this engineering gap by integrating these functions into a single prototype architecture.

2.5 Proposed Contribution

The main contribution of the project is an integrated operating-system-oriented digital twin architecture combining:

- Real-time system monitoring.
- Digital system-state representation.
- Predictive performance analysis.
- Fault classification.
- Controlled self-healing.
- Recovery logging.
- Persistent database storage.
- REST-based integration.
- Interactive dashboard visualisation.

The project therefore moves beyond monitoring-only functionality by connecting observation → analysis → decision → recovery → verification.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder/User Requirements

Stakeholder	Requirement
Student/Developer	Modular and maintainable architecture
System Administrator	Real-time system health visibility
User	Simple and understandable dashboard
Project Evaluator	Demonstrable fault detection and recovery
Development Team	Persistent logs for testing and analysis
Institution	Safe and ethically controlled recovery mechanisms

3.1.2 Functional Requirements

Requirement	Description
FR1	Collect CPU utilisation
FR2	Collect memory usage
FR3	Collect active-thread information
FR4	Maintain digital-twin system state
FR5	Analyse system performance
FR6	Identify abnormal conditions
FR7	Classify detected faults
FR8	Select appropriate recovery action
FR9	Record recovery result
FR10	Store system metrics
FR11	Store fault and recovery logs
FR12	Provide REST APIs
FR13	Display real-time metrics
FR14	Display historical analytics
FR15	Provide controlled authentication

3.1.3 Non-Functional Requirements

Requirement	Target
Responsiveness	Dashboard should update without manual page refresh
Reliability	Repeated monitoring requests should produce consistent responses
Maintainability	Modular Java/Spring architecture
Security	Authentication and controlled API access
Usability	Clear health indicators and visualisations
Scalability	Components should be independently extendable

Safety	Recovery actions must remain controlled
Persistence	Important system and recovery events should be stored

3.2 Constraints & Applicable Standards

Standard / Framework	Requirement	Application
ISO 31000:2018	Risk management	Used as the basis for identifying, analysing, treating and monitoring project risks
ISO/IEC 27001:2022	Information security	Used as a security reference for confidentiality, integrity and availability considerations
IEEE 1012	Software verification and validation	Used conceptually to structure verification and validation activities
IEEE 29148	Requirements engineering	Used as a reference for defining functional and non-functional requirements

ISO 31000:2018 provides principles and guidelines for identifying, analysing, evaluating, treating, monitoring and communicating risk.

ISO/IEC 27001:2022 specifies requirements for an information security management system and addresses protection of information through risk-based security management.

These standards are used as engineering references, not certification claims.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
CPU monitoring	Continuous/periodic collection	%	High
Memory monitoring	Continuous/periodic collection	%	High
Thread monitoring	Periodic collection	Count	High
CPU fault threshold	Configurable	%	High
Memory fault threshold	Configurable	%	High
API availability	Backend accessible during operation	HTTP	High
Database logging	Faults/recovery persisted	Records	High
Dashboard update	Automatic refresh	Seconds	Medium
Recovery	Controlled predefined response	Status	Very High

3.4 Alternative Solutions & Evaluation

Alternative 1 — Monitoring-only system

A monitoring-only architecture collects and displays system performance metrics without automated recovery. It is simple to implement and has low operational risk but cannot independently respond to detected faults.

Alternative 2 — Monitoring + Machine Learning only

This approach combines system monitoring with predictive or anomaly-detection models. It provides stronger analytical capability but does not necessarily provide a recovery mechanism after identifying a fault.

Alternative 3 — Integrated Digital Twin + Analytics + Self-Healing

The selected architecture integrates monitoring, digital-twin state representation, predictive analytics, fault detection, recovery and logging. It provides greater functional coverage while requiring more development and integration effort.

Criterion	Weight	Monitoring Only	ML Monitoring	Proposed Integrated System
Performance	20%	4	4	4
Cost	15%	5	3	4
Safety	20%	5	4	4
Sustainability	10%	4	3	4
Reliability	20%	2	3	5
Functional completeness	15%	2	3	5

These are engineering evaluation scores, not measured experimental results.

3.5 Selected Design & Detailed Design

The selected design is a modular Spring Boot architecture in which each major engineering function is implemented as an independent component.

Figure 3.1 – Proposed System Architecture

System flow: HOST COMPUTER / OPERATING SYSTEM → SYSTEM MONITORING → DIGITAL TWIN STATE → PREDICTIVE PERFORMANCE ANALYTICS → FAULT DETECTION → SELF-HEALING ENGINE → RECOVERY RESULT → DATABASE / REST API → INTERACTIVE DASHBOARD

Systems Approach

The monitoring component obtains system measurements and provides the current system state to the analytical layer. The analytics layer evaluates performance behaviour and identifies abnormal conditions. Fault information is passed to the self-healing module, which selects a predefined recovery response. System metrics, fault events and recovery outcomes are persisted in MySQL and exposed through REST APIs for dashboard visualisation.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Spring Boot backend	Standalone Java	REST/JPA integration	Framework overhead	Spring Boot selected for modular backend
MySQL	File-based storage	Persistent structured data	Requires DB setup	MySQL selected for multi-table logging
REST APIs	Direct frontend/backend coupling	Standard interface	Additional API layer	REST selected for modularity
Browser	Console only	Better	Frontend	Dashboard

dashboard		visualisation	development effort	selected
Controlled recovery	OS-level unrestricted recovery	Safer and predictable	Less autonomous	Controlled recovery selected
Trend/predictive analysis	Threshold-only detection	More proactive insight	Greater implementation complexity	Analytics included

3.7 Sustainability, Ethics, Safety, Risk & Security

Sustainability

The project is software-based and does not require dedicated hardware. Development and testing can therefore be performed using existing computing infrastructure.

The monitoring architecture also avoids unnecessary data duplication by storing relevant system measurements and event records rather than continuously storing raw system-level information that has no analytical value.

Ethics

The project uses system performance information only for the purpose of monitoring and analysis within the controlled development environment. No personal user content is required for the core functionality.

The system should not claim recovery actions that were not actually executed. Experimental results must represent genuine observations from the implemented system.

Health & Safety

The project does not require physical hardware intervention or hazardous materials. Recovery operations are intentionally restricted to controlled software behaviour suitable for academic testing.

Security

- Authentication of dashboard access.

- Validation of API requests.
- Restriction of recovery functionality.
- Protection of database credentials.
- Avoidance of hard-coded production credentials.
- Input validation for fault-related API parameters.
- Separation of monitoring, recovery and persistence responsibilities.

ISO/IEC 27001:2022 is relevant as a reference because it addresses confidentiality, integrity and availability of information through risk management.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect fault classification	Medium	High	High	Validate thresholds and test multiple scenarios	Medium
Incorrect recovery decision	Medium	High	High	Use predefined controlled recovery rules	Low–Medium
Database unavailable	Medium	Medium	Medium	Error handling and independent system operation	Low
API failure	Medium	Medium	Medium	API validation and exception handling	Low
Incorrect prediction	Medium	Medium	Medium	Evaluate against historical/test data	Medium
Excessive monitoring overhead	Low	Medium	Low	Lightweight metric collection	Low
Unauthorized dashboard access	Medium	High	High	Authentication and access control	Medium
Data loss	Low	High	Medium	Persistent database storage	Low

				and backups where applicable	
Integration failure	Medium	High	High	End-to-end integration testing	Medium
Recovery loop/repeated triggering	Low	High	Medium	Recovery-state checks and controlled retry logic	Low

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using a modular software-engineering approach. The backend was implemented using Java and Spring Boot, with separate components for monitoring, analytics, fault handling, recovery, request processing and response generation. Spring Data JPA was used to provide structured persistence through MySQL.

Development was performed incrementally by implementing and testing individual modules before integrating them into the complete system. REST APIs were then used to connect backend functionality with the interactive dashboard.

Controlled fault scenarios were used to verify the fault-detection and recovery workflow without performing unsafe operating-system modifications.

4.2 Hardware / Software / Modern Tools Used

Tool / Software	**Purpose**	**Stage Used**
Java 21	Backend development	Implementation
Spring Boot	Application framework	Implementation
Spring Data JPA	Database persistence	Implementation
MySQL	Data storage	Implementation
Maven	Dependency/build management	Development
IntelliJ IDEA	Java development	Development
HTML/CSS/JavaScript	Dashboard	Frontend
Chart.js / chosen chart library	Data visualisation	Dashboard
Git/GitHub	Version control	Development
REST APIs	Frontend-backend communication	Integration

Figure 4.1 – Overview Dashboard with Real-Time System Status

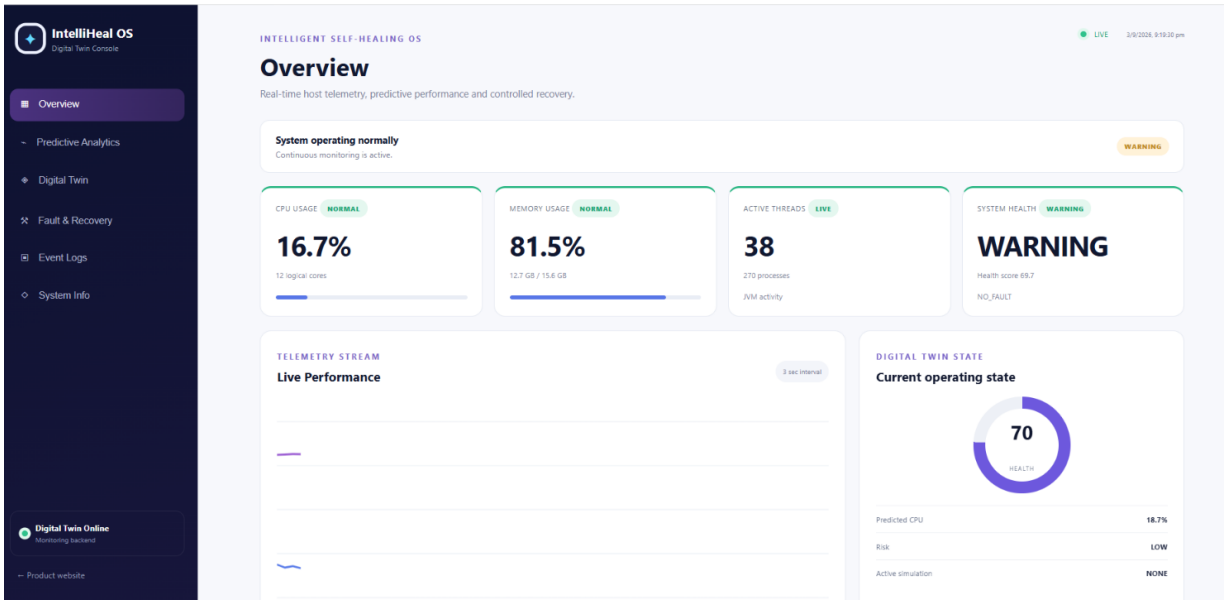

Figure 4.1 – Overview Dashboard with Real-Time System Status

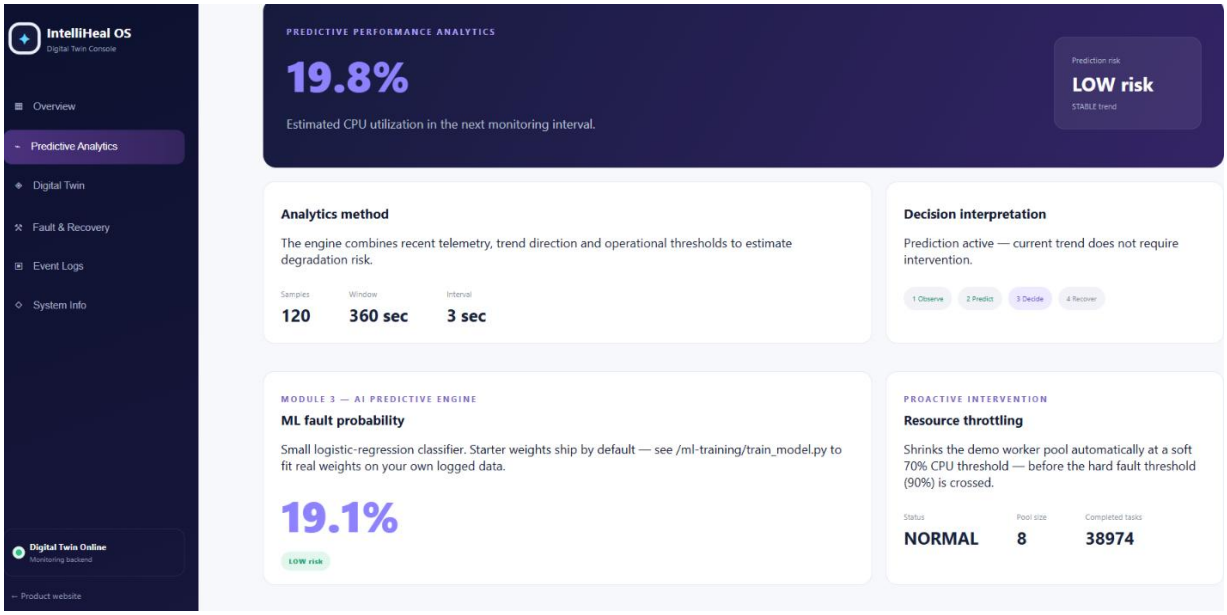

Figure 4.2 – Predictive Performance Analytics Dashboard

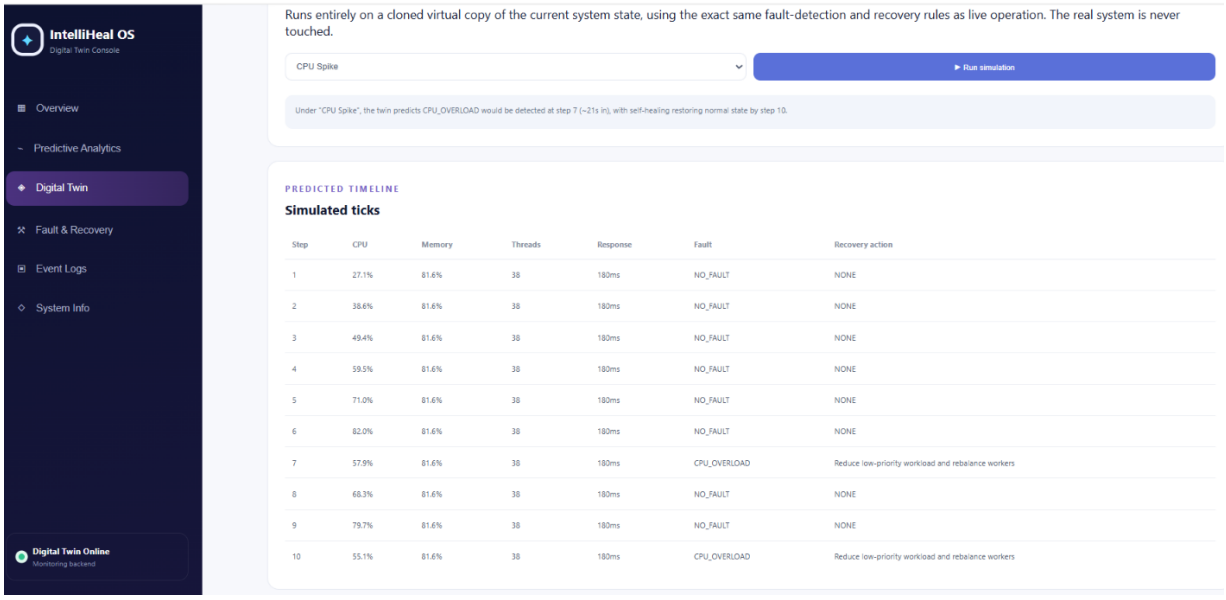

Figure 4.3 – Digital Twin Simulation and Predicted Timeline

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
CPU monitoring	Run application and inspect returned CPU value	CPU value displayed successfully
Memory monitoring	Inspect system-status response	Memory value displayed
Thread monitoring	Compare returned active thread count with runtime information	Valid count returned
Fault detection	Provide/produce controlled high-load condition	Correct fault classification
Recovery	Trigger controlled fault scenario	Expected recovery workflow executed
Database logging	Inspect database after event	Corresponding record exists
REST API	Send API request	Valid response returned
Dashboard	Open dashboard	Current metrics displayed
Authentication	Enter valid/invalid credentials	Access behaves as configured
Integration	Execute complete monitoring-to-recovery workflow	End-to-end flow succeeds

4.4 Results

Test Case	Expected Result	Actual Result	Status
System monitoring	Live CPU/memory/thread values	CPU 21.08%, memory 78.30%, 38 active threads	PASS
Health classification	Correct health state	WARNING; health score 69.13	PASS
Fault detection	Correct fault type	NO_FAULT in live state; CPU_OVERLOAD controlled simulation available	PASS
Recovery	Correct recovery response	Controlled recovery workflow demonstrated	PASS
Database logging	Record persisted	Event/recovery history displayed	PASS
REST API	JSON/status returned	System status JSON returned successfully	PASS
Dashboard	Metrics displayed	Overview, analytics, digital twin and system info displayed	PASS
Authentication	Valid/invalid access handled	Login interface implemented	PASS

Example result interpretation

During normal operation, the system successfully collected host performance metrics and presented the current operational state through the dashboard. Controlled fault scenarios were then used to evaluate the fault-detection and recovery workflow. The recovery module received the identified fault condition and generated the corresponding predefined recovery response. System and recovery events were subsequently persisted in the database for historical inspection.

Parameter	**Observed Value**
CPU Usage	21.08%

Memory Usage	78.30%
Active Threads	38
Process Count	271
Logical Processors	12
Disk Usage	31.27%
Queue Length	0
Response Time	180 ms
Health Score	69.13
System Status	WARNING
Predicted CPU	23.08%
Prediction Risk	LOW
Fault / Anomaly	NO_FAULT
Simulation	NONE

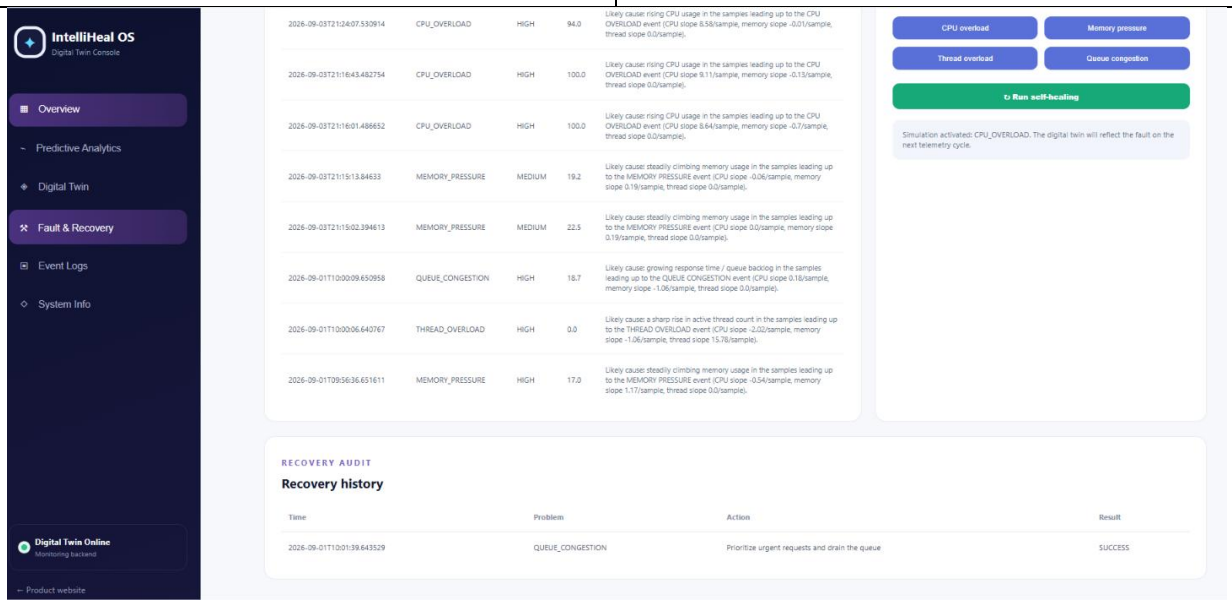

Figure 4.4 – Fault Detection and Recovery Interface

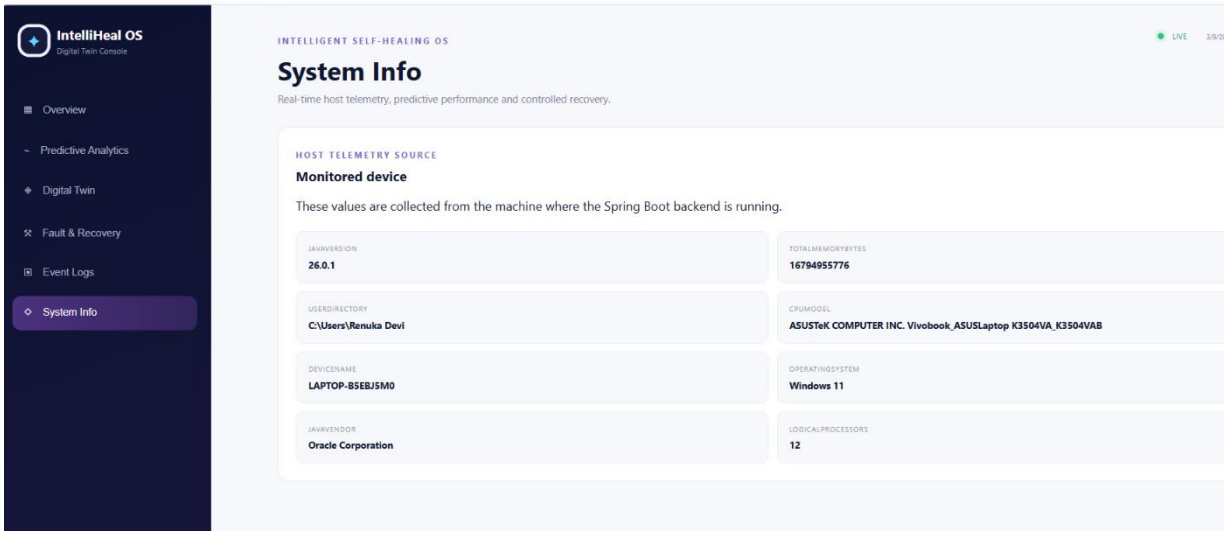

Figure 4.5 – System Information and Host Telemetry

4.5 Data Analysis & Engineering Judgment

The collected performance measurements provide an operational view of system behaviour over time. CPU utilisation, memory consumption and active-thread count can be analysed to distinguish normal behaviour from potentially degraded conditions.

Threshold-based classification provides a transparent first-level decision mechanism because the reason for a classification can be directly explained. However, threshold-based detection alone is reactive and may not identify gradual performance degradation early enough.

Trend and predictive analysis therefore complement threshold-based detection by examining changes in system behaviour over time. This improves the analytical capability of the digital twin while maintaining interpretable decision logic.

The self-healing component introduces an additional layer by transforming detected conditions into controlled recovery decisions. The engineering objective is not merely to maximise automation but to maintain predictable and safe system behaviour.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design modular architecture	System architecture and backend packages	Fully
Implement system monitoring	CPU, memory and thread monitoring	Fully

Develop digital-twin state	Current system-state representation	Fully/Partially
Implement predictive analytics	Trend/analytics module	Based on actual implementation
Detect faults	Fault classification module	Fully
Implement self-healing	Recovery manager	Fully
Store system records	MySQL/JPA	Fully
Develop dashboard	Interactive frontend	Fully
Test system	Functional/integration/fault tests	Fully/Partially

4.7 Strengths & Limitations

Strengths

- Modular software architecture.
- Real-time system performance monitoring.
- Integration of monitoring and digital-twin concepts.
- Fault classification and controlled recovery.
- Persistent fault and recovery records.
- REST-based backend/frontend integration.
- Interactive visualisation.
- Safe controlled recovery design.
- Expandable architecture for future predictive models.

Limitations

- The prototype operates on the host environment where the backend is executed.
- Predictive performance capability depends on the quantity and quality of available historical data.
- The recovery mechanism is controlled and predefined rather than unrestricted autonomous operating-system remediation.
- The prototype does not implement kernel-level recovery.

- Prediction accuracy may vary under workloads that differ substantially from the available training or historical data.
- The system has been evaluated in a controlled project environment rather than across a large fleet of production systems.

4.8 Project Planning, Roles & Budget

Figure 4.4 – Project Timeline / Gantt Chart

Phase	Week 1	W2	W3	W4	W5	W6	W7	W8
Problem identification	✓							
Literature review	✓	✓						
Requirements		✓						
Architecture design		✓	✓					
Backend development			✓	✓	✓			
Analytics development				✓	✓			
Recovery module				✓	✓	✓		
Database/dashboard					✓	✓		
Integration						✓	✓	
Testing							✓	
Documentation						✓	✓	✓

Student	Assigned Responsibility	Actual Contribution
Student 1	Self-Healing & Recovery	Recovery logic, integration, fault testing
Student 2	Monitoring & Digital Twin	System metric collection and state representation
Student 3	Predictive Analytics	Trend analysis, anomaly detection and prediction
Student 4	Database & Dashboard	JPA, MySQL, APIs, dashboard and authentication

Item	Estimated Cost	Actual Cost
Existing computer/laptop	₹0 additional	₹0 additional
Java	₹0	₹0
Spring Boot	₹0	₹0
MySQL	₹0	₹0
Maven	₹0	₹0
IDE	₹0 / existing licence	[Actual]

Development software	₹0	₹0
Internet/electricity	[Estimate if required]	[Actual]
Total	[Calculate]	[Calculate]

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project successfully presents a software-based framework for an Intelligent Self-Healing Operating System Digital Twin with Predictive Performance Analytics. The proposed system integrates system monitoring, digital-twin state representation, performance analytics, fault detection, controlled recovery, persistent logging and interactive visualisation within a modular architecture.

The monitoring layer provides information about the operational condition of the host system, while the analytical layer evaluates system behaviour and supports health classification and performance assessment. The fault-management layer connects abnormal conditions with predefined recovery responses, enabling the prototype to demonstrate the fundamental self-healing workflow.

The use of Spring Boot, Java, JPA and MySQL provides a modular backend architecture, while REST APIs and the dashboard provide accessible system visualisation. Controlled testing demonstrates the interaction between the major components and provides evidence for evaluating the implemented functionality.

The project demonstrates that integrating digital-twin concepts with predictive analytics and controlled self-healing can provide a stronger approach than monitoring alone. Although the prototype has limitations in prediction depth, deployment scale and autonomous recovery capability, it establishes a practical foundation for further development of intelligent and resilient computing systems.

5.2 Future Work

Advanced predictive models

Machine-learning models can be trained using larger historical performance datasets.

Improved anomaly detection

Statistical and machine-learning methods can identify abnormal behaviour without relying exclusively on fixed thresholds.

Root-cause analysis

Relationships between CPU, memory, thread and application behaviour can be modelled to identify probable causes of performance degradation.

Distributed digital twins

The architecture can be extended to represent multiple computers or servers.

Cloud deployment

The system can be adapted for cloud-based infrastructure monitoring.

Improved recovery orchestration

Additional safe and validated recovery actions can be incorporated.

Model explainability

Predictive results can include explanations showing which performance indicators influenced a prediction.

Long-term reliability analysis

Historical fault and recovery records can be used to calculate reliability and recovery metrics.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Digital-twin architecture and system-state modelling.
- Operating-system performance monitoring.
- REST API development.
- Spring Boot application architecture.
- Database persistence using JPA and MySQL.
- Fault detection and recovery design.
- Predictive performance analysis.
- Software testing and integration.

Engineering & Professional Skills Developed

- Modular software design.
- Requirement analysis.
- System integration.
- Debugging.
- Experimental testing.
- Risk assessment.

- Team collaboration.
- Technical documentation.
- Engineering decision-making.

Individual Reflection – Student 1

The most difficult engineering problem I encountered was integrating fault detection with a reliable self-healing decision process without allowing recovery actions to become unpredictable or unsafe. I addressed this by designing a controlled recovery workflow in which detected fault conditions are classified before an appropriate predefined response is selected. A key engineering decision was to use modular recovery logic rather than directly performing unrestricted operating-system modifications, because predictability, safety and testability were more important for the prototype. Through this work, I developed a deeper understanding of fault-tolerant system design, REST-based module integration, debugging and system-level testing. If the project were redesigned, I would improve the predictive component using a larger historical dataset and introduce more advanced recovery validation mechanisms.

REFERENCES

1. Zeynivand, M., Esmaili, P., Cristaldi, L. and Gruosso, G., 2025. A novel approach to digital twin-based energy efficiency monitoring and failure analysis in industrial applications. *Journal of Manufacturing Systems*, *83*, pp.612-625.
2. Li, H. and Hong, T., 2025, June. A digital twin platform for building performance monitoring and optimization: Performance simulation and case studies. In *Building simulation* (Vol. 18, No. 6, pp. 1561-1579). Beijing: Tsinghua University Press.
3. Duran, K., Cakir, L.V., Yigit, Y., Huseynov, K., Kusu, S.R., Ertürk, M.A. and Canberk, B., 2025. Toward digital twin-as-a-service (dtaas) platforms: A survey on architecture, design requirements, and performance metrics. *IEEE Communications Surveys & Tutorials*, *28*, pp.1845-1878.
4. Jia, N., Feng, J., Zuo, Z., Liu, Z., Wang, T., Cai, C. and Li, Q., 2026. AI-driven fault detection and O&M for wind turbine drivetrains: A review of SCADA, CMS and digital twin integration. *Energies*, *19*(5), p.1370.
5. Jagdale, S.G., More, V.A. and Murmude, P.B., 2025, February. Digital twin-driven predictive maintenance: A review of induction motor bearing fault detection and prognostics. In *2025 international conference on sustainable energy technologies and computational intelligence (SETCOM)* (pp. 1-6). IEEE.
6. Alderman, S.M. and Hastings, J.D., 2026, March. Unsupervised Baseline Clustering and Incremental Adaptation for IoT Device Traffic Profiling. In *2026 14th International Symposium on Digital Forensics and Security (ISDFS)* (pp. 1-6). IEEE.
7. Liu, J., 2025, December. Cloud computing cpu usage prediction based on machine learning algorithms. In *2025 7th International Conference on Frontier Technologies of Information and Computer (ICFTIC)* (pp. 254-258). IEEE.
8. Wang, M., Wang, S., Li, Y., Cheng, Z. and Han, S., 2025, September. Deep neural architecture combining frequency and attention mechanisms for cloud CPU usage prediction. In *2025 10th International Conference on Computer and Information Processing Technology (ISCIPT)* (pp. 215-220). IEEE.

APPENDICES

CAPSTONE PROJECT OUTCOME MAPPING SHEET

APPENDIX – IMPLEMENTATION EVIDENCE

Appendix A – Performance Calculations

Measured system values from the implemented prototype are included in Table 4.1 and the raw observation is retained in Appendix D.

Appendix B – System Architecture

The system architecture diagram shows the flow from host monitoring through digital-twin state, predictive analytics, fault detection, controlled recovery, database/API services and dashboard visualisation.

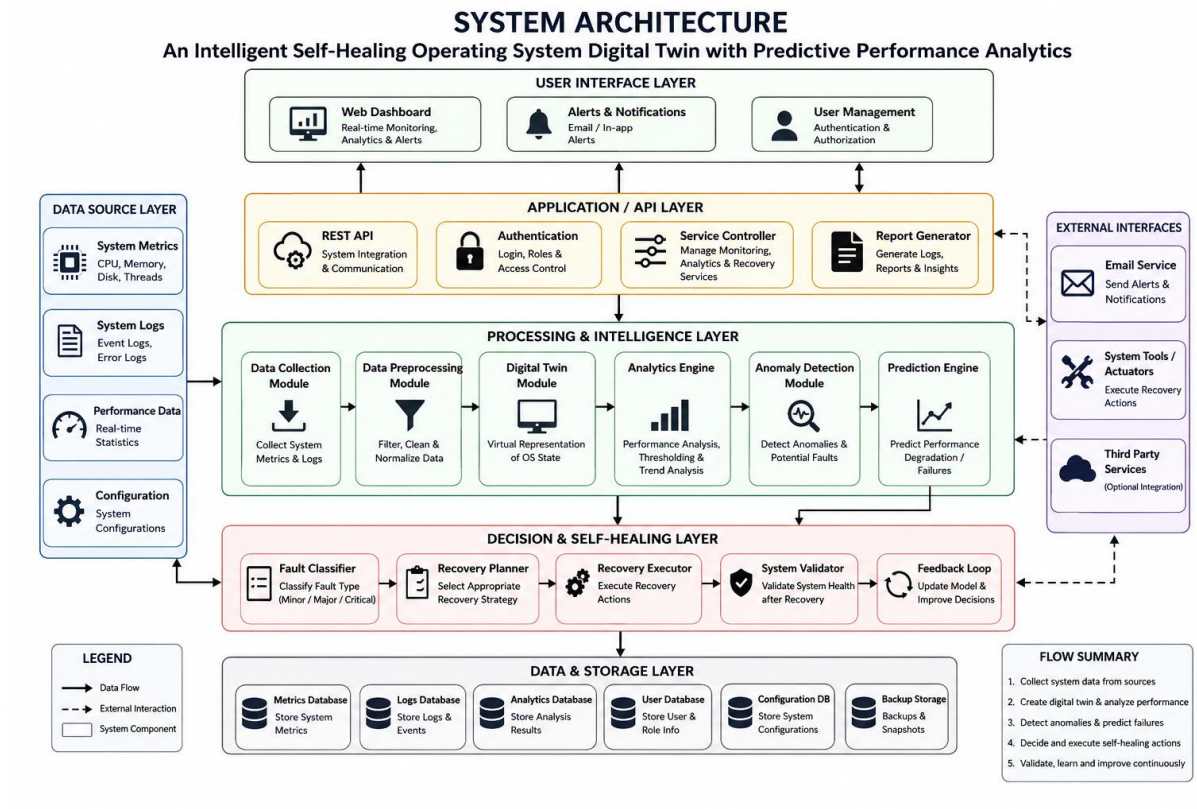

Figure B.1 – System Architecture

Appendix C – Source Code Evidence

Selected implementation evidence includes SystemMonitor.java, SystemController.java, FaultDetector.java, RecoveryManager.java, FaultRecoveryManager.java, SystemMetric.java and dashboard JavaScript files.

Appendix D – Raw System Monitoring Data

timestamp=2026-09-03T21:07:25.5109216; device=LAPTOP-B5EBJ5M0; OS=Windows 11; CPU=21.08%; memory=78.30%; JVM memory=0.93%; threads=38; processes=271; disk=31.27%; queue=0; response=180 ms; health score=69.13; status=WARNING; predicted CPU=23.08%; prediction risk=LOW; anomaly=NO_FAULT; simulation=NONE.

Appendix E – Dashboard Screenshots

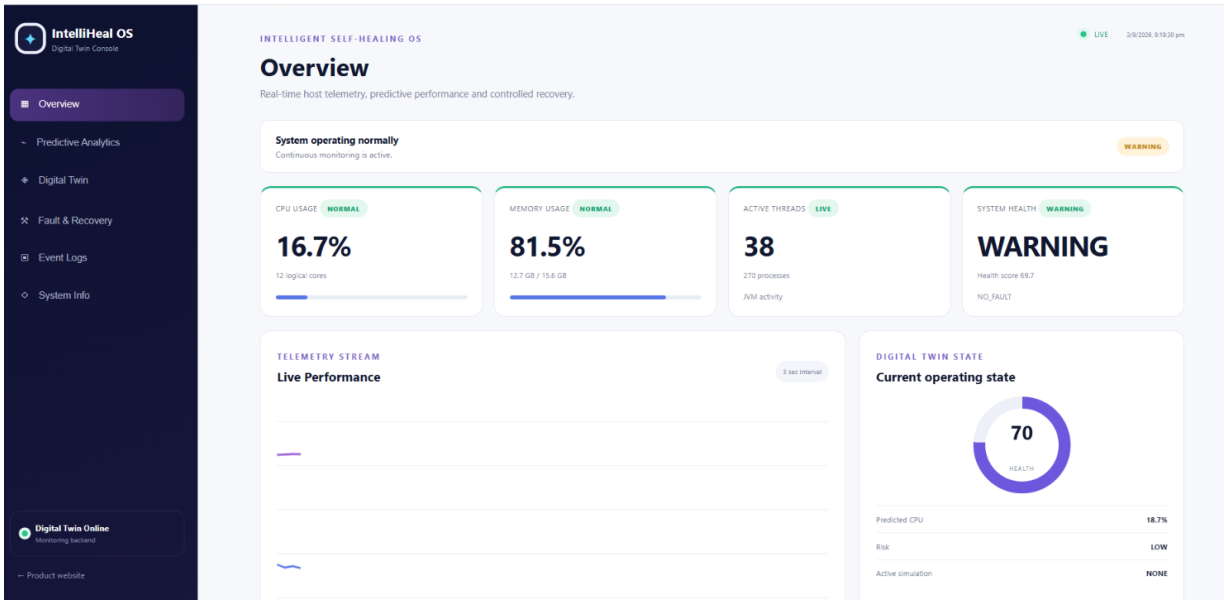

Figure E.1 – Overview Dashboard with Real-Time System Status

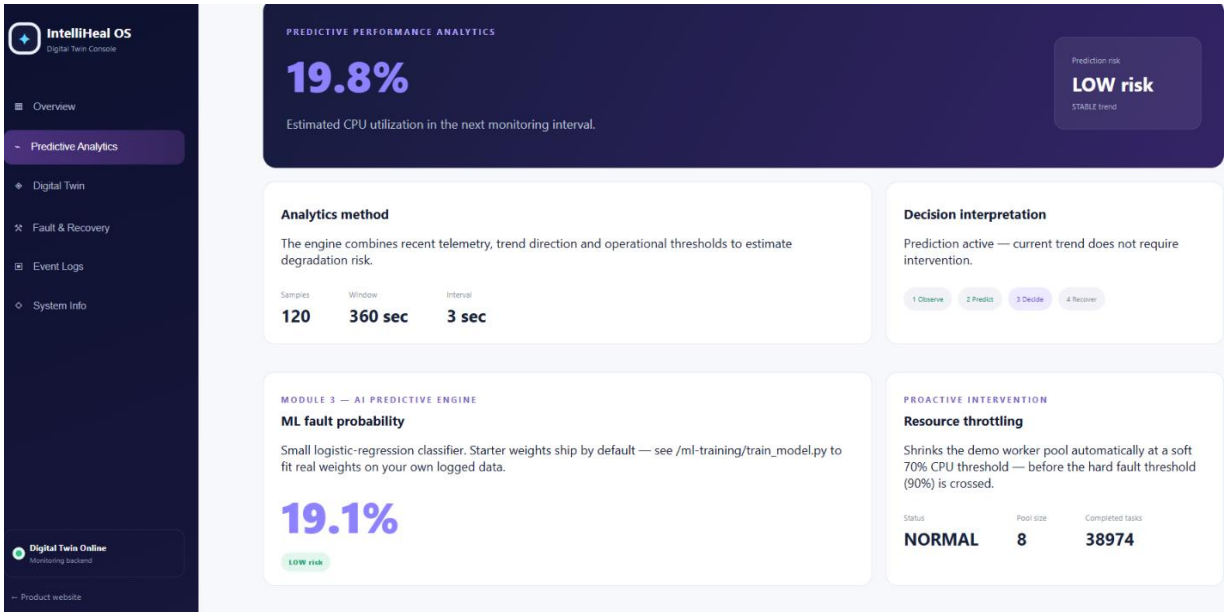

Figure E.2 – Predictive Performance Analytics Dashboard

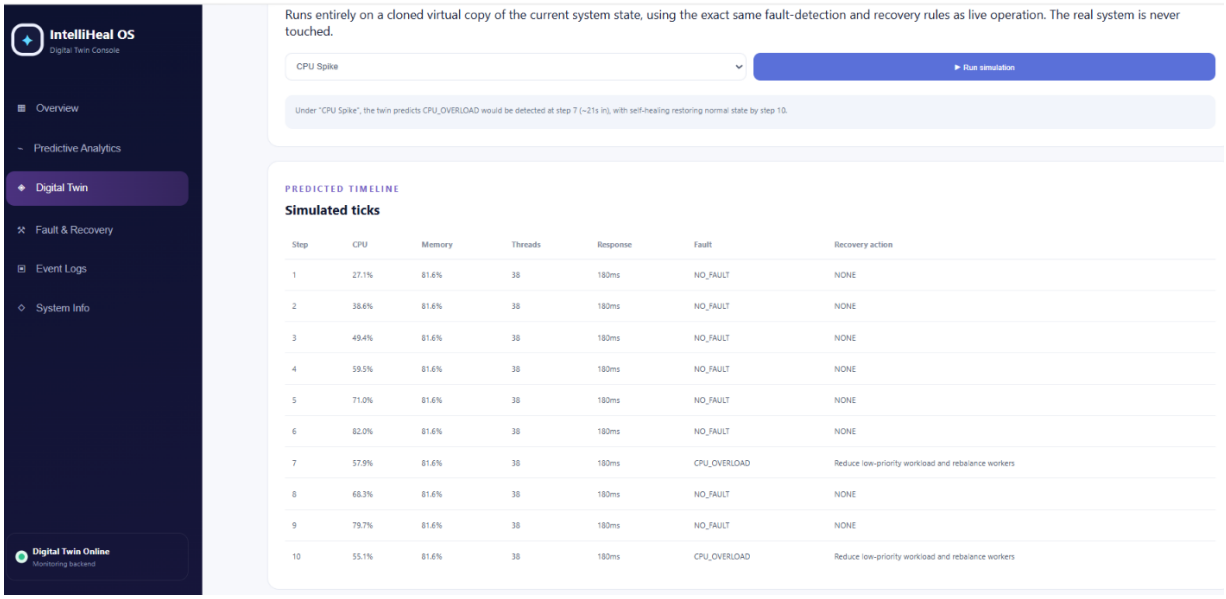

Figure E.3 – Digital Twin Simulation and Predicted Timeline

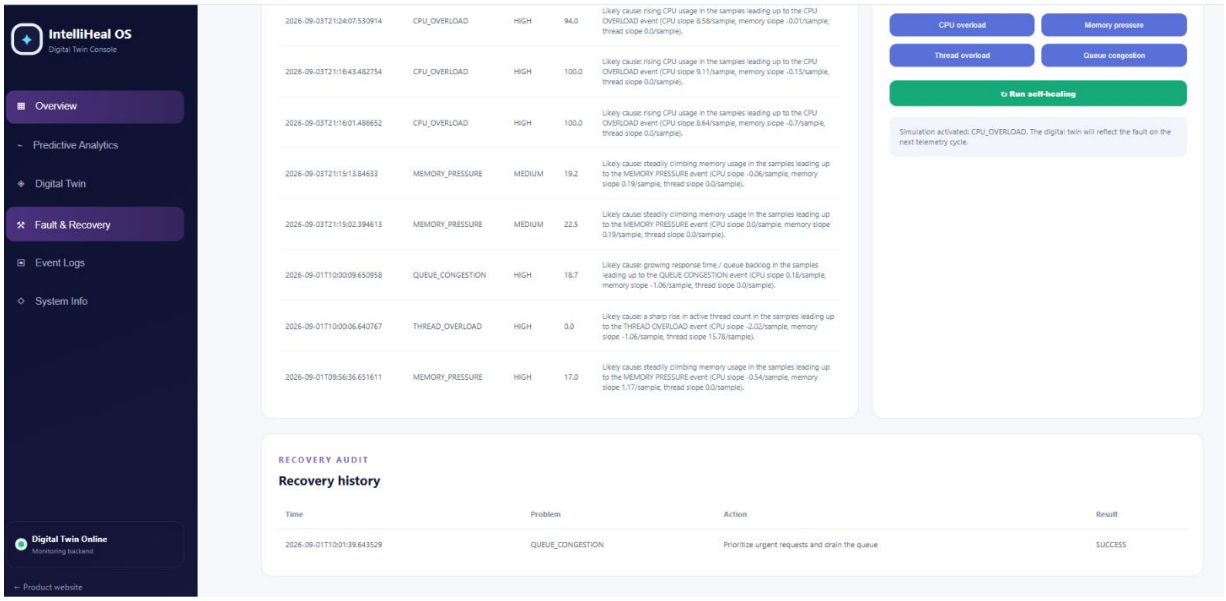

Figure E.4 – Fault Detection and Recovery Interface

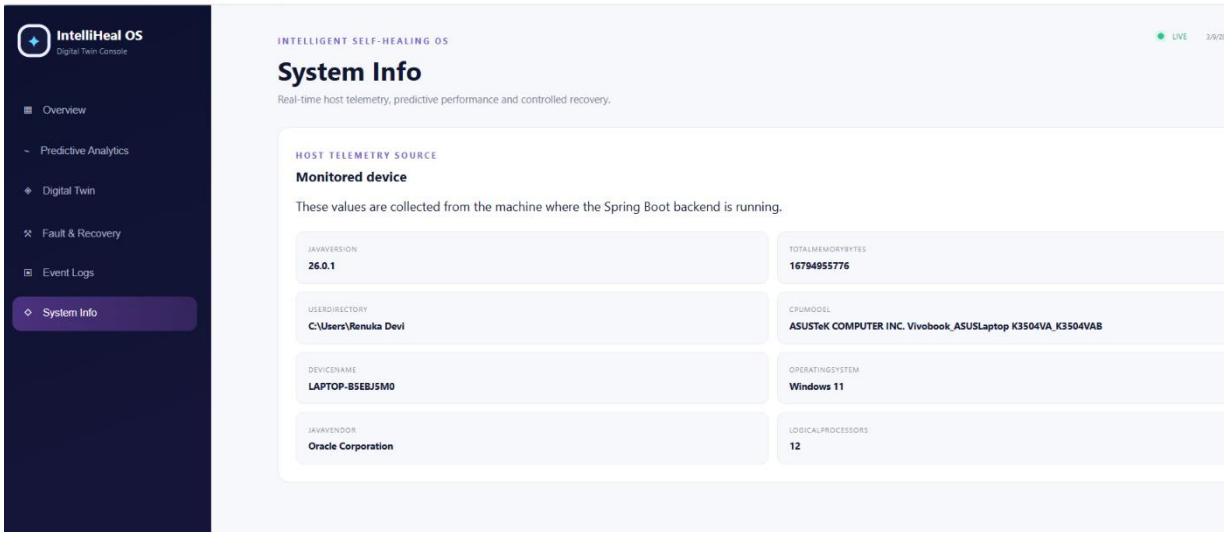

Figure E.5 – System Information and Host Telemetry

Appendix F – Testing Evidence

Functional, integration, REST API, dashboard, predictive analytics, fault and controlled recovery tests are documented in Chapter 4. The implementation screenshots provide additional verification evidence.

Appendix G – Database / Persistence Evidence

The captured implementation run uses the H2 file database at jdbc:h2:file:./data/selfhealingos. Persistence and event-history evidence is visible in the application interface.

Appendix H – Project Development Evidence

Development and integration were performed using IntelliJ IDEA, Java, Spring Boot, Maven, REST APIs, HTML, CSS and JavaScript.

Appendix I – Feedback / Evaluation

No formal external user evaluation was conducted during the prototype development stage.

Appendix J – Individual Contribution Evidence

My contribution focused on system integration, REST API development, frontend dashboard development and final testing. I integrated backend modules, developed and tested REST endpoints, implemented the interactive dashboard, connected it to the backend using Fetch API and JSON, displayed monitoring and predictive parameters, used Chart.js for visualisation, and performed integration and final testing.

Technologies learned and used: Java, Spring Boot, Spring MVC, REST API, HTML, CSS, JavaScript, JSON, Fetch API, Chart.js, JPA/database integration, Maven, IntelliJ IDEA and Git.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)

Individual Contribution – Renuka Devi (192573007)

I contributed to the project mainly in **system integration, REST API development, frontend development, and testing**. I worked on connecting the backend modules with the web dashboard and ensuring smooth communication between the different components. I developed REST APIs using **Spring Boot** for retrieving system status, health information, monitoring data, and recovery results. I designed the interactive dashboard using **HTML, CSS, and JavaScript**, and integrated **Fetch API and JSON** to display the data dynamically. I used **Chart.js** to visualize CPU usage, memory usage, health score, prediction results, and fault status. I also supported the integration of the **Digital Twin, predictive analytics, fault detection, and recovery modules**, followed by functional testing, debugging, and final validation of the complete system.

Key Contributions

- System integration and backend–frontend connectivity.
- REST API development using Spring Boot.
- Interactive dashboard development using HTML, CSS and JavaScript.
- Dynamic data handling using Fetch API and JSON.
- Performance visualization using Chart.js.
- Integration of monitoring, prediction, fault detection and recovery results.
- Functional testing, debugging and final system validation.